

Advanced Load Balancer Concepts

Do you see any problem with this current approach?

What did I tell you about this load balancer? This load balancer is also a server, right? Yes, it is also a server, which means **it can also crash**.

If I have only a single load balancer, can't it fail?

All the requests from millions of users are coming to this one load balancer. It is also just a machine. No matter how much I vertically scale it by increasing its RAM, CPU, and other resources, there will eventually be a limit. After a certain point, this load balancer can also crash.

So, how can we protect ourselves from this problem?

One approach is to create multiple load balancers.

But then another question arises:

Which load balancer should the client call?

Suppose I have three load balancers:

- Load Balancer 1
- Load Balancer 2
- Load Balancer 3

Which one should the user connect to?

We could put another load balancer in front of these three load balancers and tell users:

"You call this load balancer, and it will decide which load balancer you should go to."

But then what happens?

The load on this front load balancer increases.

And this load balancer can also crash.

So how do we solve this problem?

Multiple Load Balancers in Different Regions

We can have different load balancers for different geographical regions.

For example:

- One load balancer for **India**
- One load balancer for **America**
- One load balancer for **Europe**

Now suppose you are a user located in India.

When you send a request, how does your system know which load balancer to connect to?

The answer is **DNS**.

Your DNS can return the IP address of the load balancer that is located in India.

Therefore, requests from users in India can be directed to the Indian load balancer.

Similarly:

- Requests from America can go to the American load balancer.
- Requests from Europe can go to the European load balancer.

Who decides this?

DNS.

Your ISP provides access to DNS infrastructure, and DNS can direct your request toward the appropriate regional endpoint.

So, when you make a request, DNS can effectively tell you:

"Go to the Indian load balancer."

Your request then reaches the Indian load balancer, which forwards it to the appropriate backend server.

What If the Indian Load Balancer Fails?

Now you might ask:

"What happens if the Indian load balancer itself fails?"

The DNS infrastructure can monitor the health of the regional load balancers.

If the Indian load balancer is detected as unavailable, traffic can be directed toward another available load balancer, such as the one in America.

Each load balancer is a separate machine or infrastructure endpoint, so each can have a different IP address.

Therefore, we can have different regional load balancers with different addresses.

This is one strategy for avoiding a **single point of failure** at the load-balancer layer.

Anycast

There is another strategy that is commonly used.

It is called **Anycast**.

In Anycast, we can deploy many load balancers across different geographical locations.

For example, imagine we have hundreds of load balancers distributed around the world.

The important thing is that they can advertise the **same IP address**.

For example:

India	→ 1.1.1.2
Europe	→ 1.1.1.2
America	→ 1.1.1.2
Pakistan	→ 1.1.1.2
Iran	→ 1.1.1.2
Other	→ 1.1.1.2

This might seem strange.

Normally, people think:

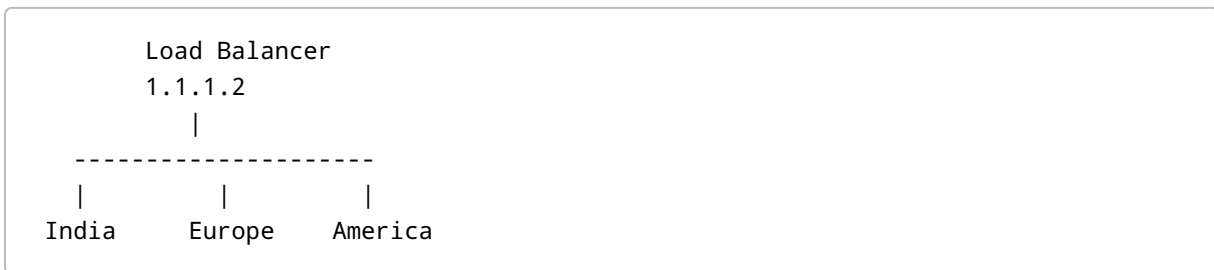
"Multiple machines cannot have the same IP address."

But with **Anycast**, the same IP address can be advertised from multiple locations.

Large infrastructure providers such as Cloudflare and Google use Anycast-style networking extensively.

How Does Anycast Decide Where the Request Goes?

Suppose the same IP address is available from multiple locations:



You might wonder:

"If all of them have the same IP address, which load balancer will receive my request?"

The answer is based on **network routing**.

When your DNS gives you the IP address, your request does not travel directly to that physical machine.

There are many routers between you and the destination.

Those routers determine the best route to the destination.

As a result, your traffic is generally routed toward the appropriate/nearest Anycast location according to the routing topology.

For example:

- An Indian user may reach the Indian location.
- A European user may reach the European location.
- An American user may reach the American location.

And if one location becomes unavailable, routing can converge toward another available location.

This is called **Anycast**.

Anycast vs Regional DNS

There are differences between these two approaches.

Regional DNS approach

With regional DNS, you can configure DNS so that users are directed toward different regional load balancers.

For example:

```
India    → Indian Load Balancer
Europe  → European Load Balancer
America → American Load Balancer
```

This approach can be comparatively simpler and less expensive to implement.

Anycast approach

With Anycast, the same IP address is advertised from multiple locations.

The network routing infrastructure decides where the traffic should go.

Therefore, Anycast requires significantly more sophisticated networking infrastructure.

The routing layer needs to understand and manage these advertisements and routes.

So the infrastructure requirements are much higher.

The Problem With Simply Choosing the Lowest Response Time

Now let's consider another problem.

Suppose we want to create a **smart load balancer**.

A simple strategy might be:

"Send every request to the server that currently has the lowest response time."

For example:

Server A → 3 seconds
Server B → 5 seconds
Server C → 8 seconds

Obviously, Server A looks like the best choice.

So whenever a new request arrives, we send it to Server A.

But is this really the best strategy?

Not necessarily.

Why Response Time Alone Is Not Enough

Let's take an example similar to YouTube.

Suppose users are uploading videos.

Some videos are:

- Short videos
- Medium-length videos
- Very long videos

Now imagine that our servers currently have the following workloads.

Server A

Current response time:

3 seconds

It looks like the fastest server.

But suppose it already has **5 requests waiting**, and those requests represent a total of **50 hours of video processing**.

Server B

It currently has several requests, perhaps representing around **7 hours of work**.

Server C

It has several requests, but the total amount of work is only around **3 minutes**.

Now a new request arrives.

If we only look at historical response time, we might say:

"Server A responds in 3 seconds, so let's send the request there."

But Server A already has a huge workload.

The new request could end up waiting for a very long time.

Meanwhile, Server C may have a higher historical response time, but its current workload is extremely small.

So Server C might actually complete the new request much faster.

This shows an important principle:

We should not make load-balancing decisions only from historical response time.

We also need to understand the **current workload and capacity** of each server.

Making the Load Balancer Smarter

A smarter load balancer should consider:

- Current number of requests
- Current workload
- Estimated processing time
- Server capacity
- CPU utilization
- Memory utilization
- Queue length
- Historical performance
- Type/size of the request

Instead of asking only:

"Which server responded fastest in the past?"

we should ask:

"Which server can complete this particular request most efficiently given its current workload?"

For example, suppose we receive a request to process a **one-hour video**.

We might estimate:

```
Server A → 1.3 hours  
Server B → 2 hours  
Server C → 30 minutes
```

Even if Server A historically has the lowest response time, Server C may be the better choice because it currently has more available capacity.

Historical Performance vs Current Capacity

This is an important distinction.

If we use only the **least-response-time strategy**, we are mostly looking at historical information.

For example:

```
Server A → historically 3 seconds  
Server B → historically 5 seconds  
Server C → historically 8 seconds
```

Based on this information, we might continuously send traffic to Server A.

But then Server A becomes overloaded.

Therefore, we need to consider **current capacity**, not just historical performance.

The load balancer should estimate:

"How long will the requests currently running on each server take to finish?"

Then it can make a better routing decision.

Modern Load Balancers

Modern architectures can make load balancing much more intelligent.

The load balancer does not necessarily behave like a simple device that blindly forwards packets.

Depending on the layer and architecture, it can inspect request information and use it to make routing decisions.

For example, in a video-upload scenario, it may need to understand information such as:

- What type of request is this?
- How large is the upload?
- How much processing might it require?
- How long might it take?
- What is the current workload of each server?
- Which server currently has enough capacity?

Based on these factors, the load balancer can select an appropriate backend.

Another Important Problem: Large Requests

Now let's take the example of a user uploading a **one-hour video**.

Does the entire one-hour video necessarily arrive at the load balancer as one giant piece of data?

No.

Network data is transmitted as smaller units/packets.

Conceptually, we can imagine the video arriving like this:

P1 → P2 → P3 → P4 → ...

The entire video does not necessarily arrive at once.

Instead, the data is transmitted progressively.

Therefore, the load balancer has another important responsibility.

Packet Affinity / Consistent Routing

Suppose the first packet of a video upload arrives:

P1

The load balancer decides:

"This upload should be processed by Server A."

So:

P1 → Server A

Now the next packet arrives:

P2

The load balancer must understand that **P2 belongs to the same upload/session as P1.**

Therefore:

P1 → Server A
P2 → Server A
P3 → Server A
P4 → Server A

It should not suddenly do this:

P1 → Server A
P2 → Server B
P3 → Server C
P4 → Server A

because the backend may need to combine/process the pieces of the same upload together.

Therefore, once the load balancer decides where a particular flow/session should go, subsequent packets belonging to that flow need to be routed consistently to the appropriate backend.

This is where concepts such as **session affinity/sticky sessions, connection tracking, and flow-aware routing** become important.

Final Takeaway

So, today we learned several important ideas about load balancers.

1. A single load balancer can become a Single Point of Failure

Even the load balancer itself is a server/infrastructure component and can fail.

2. Multiple regional load balancers can help

For example:

India	→ India Load Balancer
America	→ US Load Balancer
Europe	→ Europe Load Balancer

DNS can help direct users toward the appropriate regional endpoint.

3. Anycast provides another approach

Multiple locations can advertise the same IP address, while network routing determines an appropriate destination.

4. Lowest response time isn't always the best strategy

A server may have historically low latency but currently be overloaded.

5. Current workload matters

A smarter load balancer should consider:

Current workload
+
Queue length
+
Server capacity
+
Estimated processing time

+
Request characteristics

6. Large requests arrive progressively

A large video upload is not necessarily delivered as one giant block.

Data is transmitted in smaller packets/segments.

7. Consistent routing is important

If packet P1 of an upload is sent to Server A, subsequent packets belonging to the same flow should generally continue to reach the appropriate server so that the complete request can be processed correctly.

This gives us a much more realistic picture of a **modern, intelligent load-balancing architecture**, rather than simply:

```
Client
  ↓
Load Balancer
  ↓
Server
```

In real-world distributed systems, the load-balancing layer itself needs to be **highly available, geographically distributed, routing-aware, workload-aware, and capable of making intelligent traffic decisions**.